



Atelier Python – Application modulaire de gestion de tâches (To-Do List)



Par Robert DIASSÉ

Objectif : Créer une application de gestion de tâches modulaire, en ligne de commande, manipulant des fichiers, des listes et des erreurs.

Étape 1 : Création d'un module de gestion des tâches

Fichier : `gestion_taches.py`

Contenu initial :

```
# gestion_taches.py
def ajouter_tache(liste, tache):
    liste.append(tache)
    return liste

def afficher_taches(liste):
    for i, t in enumerate(liste):
        print(f"{i+1}. {t}")

def supprimer_tache(liste, indice):
    try:
        del liste[indice]
        return True
    except IndexError:
        return False
```

Comprendre que :

- `ajouter_tache`: ajoute une tâche à une liste existante.
 - `afficher_taches`: affiche les tâches avec leur index.
 - `supprimer_tache`: supprime une tâche si l'indice est valide, sinon retourne `False`.
-

Étape 2 : Création du script principal

Fichier : `main.py`

```
import gestion_taches

def menu():
    print("\n--- MENU ---")
    print("1. Ajouter une tâche")
    print("2. Afficher les tâches")
    print("3. Supprimer une tâche")
    print("4. Quitter")

def run():
    taches = []
    while True:
        menu()
        choix = input("Votre choix : ")

        if choix == "1":
            tache = input("Entrez une nouvelle tâche : ")
            gestion_taches.ajouter_tache(taches, tache)
        elif choix == "2":
            gestion_taches.afficher_taches(taches)
        elif choix == "3":
            try:
                index = int(input("Numéro de la tâche à supprimer : ")) - 1
                if not gestion_taches.supprimer_tache(taches, index):
                    print("Tâche introuvable.")
            except ValueError:
                print("Veuillez entrer un nombre valide.")
        elif choix == "4":
            print("Fermeture du programme.")
            break
        else:
            print("Choix invalide.")
```

Étape 3 : Ajout d'un module utilitaire `fichier.py` pour sauvegarder et charger les tâches

```
# fichier.py
def sauvegarder(taches, nom_fichier):
    with open(nom_fichier, "w") as f:
        for tache in taches:
            f.write(tache + "\n")

def charger(nom_fichier):
    try:
        with open(nom_fichier, "r") as f:
            return [ligne.strip() for ligne in f.readlines()]
    except FileNotFoundError:
        return []
```

Modifications dans `main.py` :

```
import gestion_taches
import fichier

def run():
    fichier_nom = "mes_taches.txt"
    taches = fichier.charger(fichier_nom)
    while True:
        menu()
        choix = input("Votre choix : ")
        if choix == "1":
            tache = input("Entrez une nouvelle tâche : ")
            gestion_taches.ajouter_tache(taches, tache)
        elif choix == "2":
            gestion_taches.afficher_taches(taches)
        elif choix == "3":
            try:
                index = int(input("Numéro de la tâche à supprimer : ")) - 1
                if not gestion_taches.supprimer_tache(taches, index):
                    print("Tâche introuvable.")
            except ValueError:
                print("Erreur : veuillez entrer un entier.")
        elif choix == "4":
            fichier.sauvegarder(taches, fichier_nom)
            print("Tâches sauvegardées. À bientôt !")
            break
```

Étape 4 : Ajout de la gestion des erreurs avec des fonctions dédiées

Ajoutez un module `verificateur.py` :

```
# verificateur.py
def verifier_choix(choix):
    return choix in ["1", "2", "3", "4"]

def verifier_tache(tache):
    return len(tache.strip()) > 0
```

Utilisation dans `main.py` :

```
import verificateur

# ...
if choix == "1":
    tache = input("Entrez une nouvelle tâche : ")
    if verificateur.verifier_tache(tache):
```

```
gestion_taches.ajouter_tache(taches, tache)
else:
    print("Erreur : tâche vide non autorisée.")
```

Étape 5 : Utilisation d'un décorateur pour journaliser l'exécution

Ajoutez à `decorateurs.py` :

```
# decorateurs.py
def log_execution(fonction):
    def wrapper(*args, **kwargs):
        print(f"[LOG] Début : {fonction.__name__}")
        resultat = fonction(*args, **kwargs)
        print(f"[LOG] Fin : {fonction.__name__}")
        return resultat
    return wrapper
```

Puis dans `gestion_taches.py`, utilisez :

```
from decorateurs import log_execution

@log_execution
def ajouter_tache(liste, tache):
    liste.append(tache)
    return liste
```

Résultat attendu

Un programme en ligne de commande permettant à l'utilisateur de :

- Ajouter une tâche
- Voir les tâches
- Supprimer une tâche
- Sauvegarder automatiquement les tâches à la fermeture
- Gérer proprement les erreurs de saisie et de fichier
- Organiser le code de façon modulaire (4 modules)

Pistes d'amélioration que vous devrais explorer vous-même :

1. Ajouter une date d'échéance à chaque tâche.
2. Pouvoir trier les tâches par ordre alphabétique ou d'échéance.
3. Marquer une tâche comme "terminée".
4. Créer une interface graphique avec Tkinter à partir du programme.

5. Ajouter une interface de filtrage (terminées vs non terminées).
6. Créer des tests unitaires pour le module `gestion_taches`.